

Artificial Intelligence and Machine Learning

Neural Networks

Lecture Outline

- Classification Metrics
- Neural Networks
 - Forward pass
 - Backward pass

A century



Classification Metrics

- **Accuracy:**

Accuracy measures the proportion of correctly classified samples out of the total number of samples.

$$\text{Accuracy} = \frac{\text{Correct Samples}}{\text{All Sample}}$$

Could also be written as:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Where:

TP: True Positives

TN: True Negatives

FP: False Positives

FN: False Negatives

99%

$$\frac{198}{200} = 0.99$$

Question: Why not optimizing for the accuracy directly instead of using LogLoss?
Non-differentiable

97	3
5	95

1	0
1	TP
0	FP
0	FN
0	TN

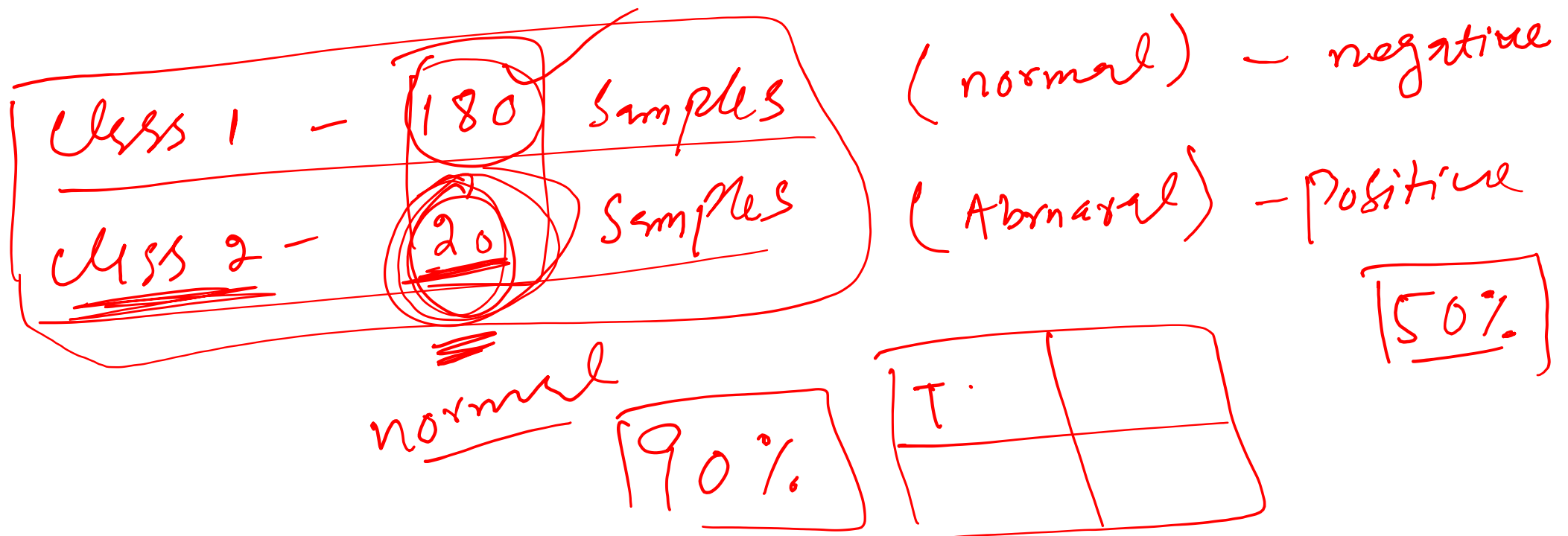
Confusion

Classification Metrics

But accuracy isn't always enough!

Imagine an imbalanced dataset with 95% negative labels (e.g., "not spam").

A model predicting "negative" all the time will be 95% accurate but useless.



Classification Metrics



But accuracy isn't always enough!

Imagine an imbalanced dataset with 95% negative labels (e.g., "not spam").
A model predicting "negative" all the time will be 95% accurate but useless.

Different goals require different metrics

- Do you care more about **catching all positives** (e.g., cancer detection)?
- Or avoiding **false alarms** (e.g., fraud detection)?

Classification Metrics



- **Precision:**

Among all the positive predictions the model made, how many are actually correct?
important when **minimizing false alarms** is critical, like in fraud detection or spam filtering.

A hand-drawn diagram of a confusion matrix, enclosed in a red rounded rectangle. It is a 2x2 grid with 'TP' (True Positive) in the top-left, 'FP' (False Positive) in the top-right, 'FN' (False Negative) in the bottom-left, and 'TN' (True Negative) in the bottom-right. The labels are underlined and the entire diagram is circled in red.

TP	FP
FN	TN

$$\text{Precision} = \frac{\text{True Positives (TP)}}{\text{True Positives (TP)} + \text{False Positives (FP)}}$$

class 1

Classification Metrics



- **Recall:**

Among all the actual positives, how many did the model correctly identify?

important when **catching all positives** is vital, such as in cancer detection.

$$\text{Recall} = \frac{\text{True Positives (TP)}}{\text{True Positives (TP)} + \text{False Negatives (FN)}}$$

Classification Metrics



- **F1 Score:**

What if both precision and recall are important?

Take their (harmonic) mean.

$$F1\ score = \frac{2}{\frac{1}{\textit{Precision}} + \frac{1}{\textit{Recall}}}$$

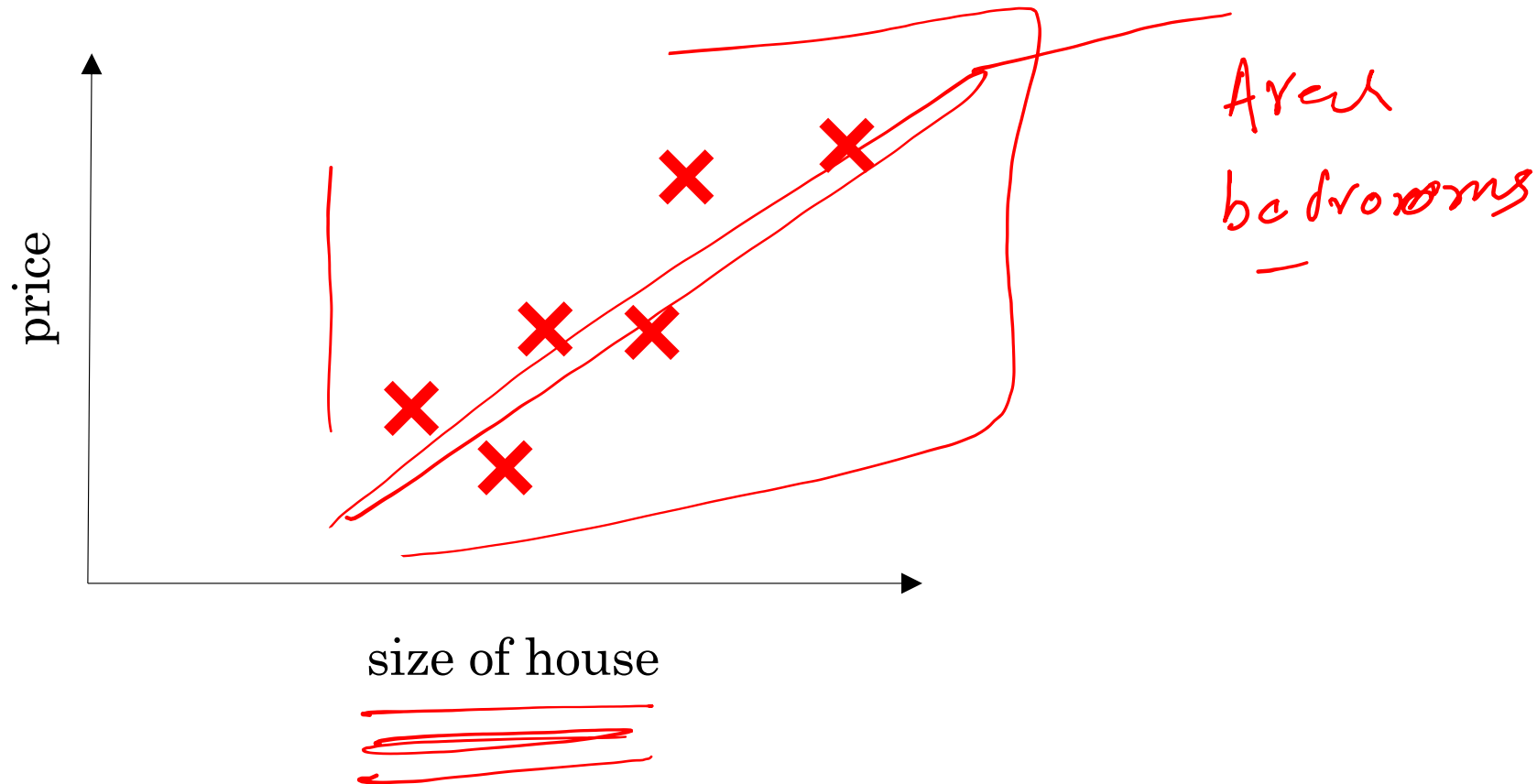


deeplearning.ai

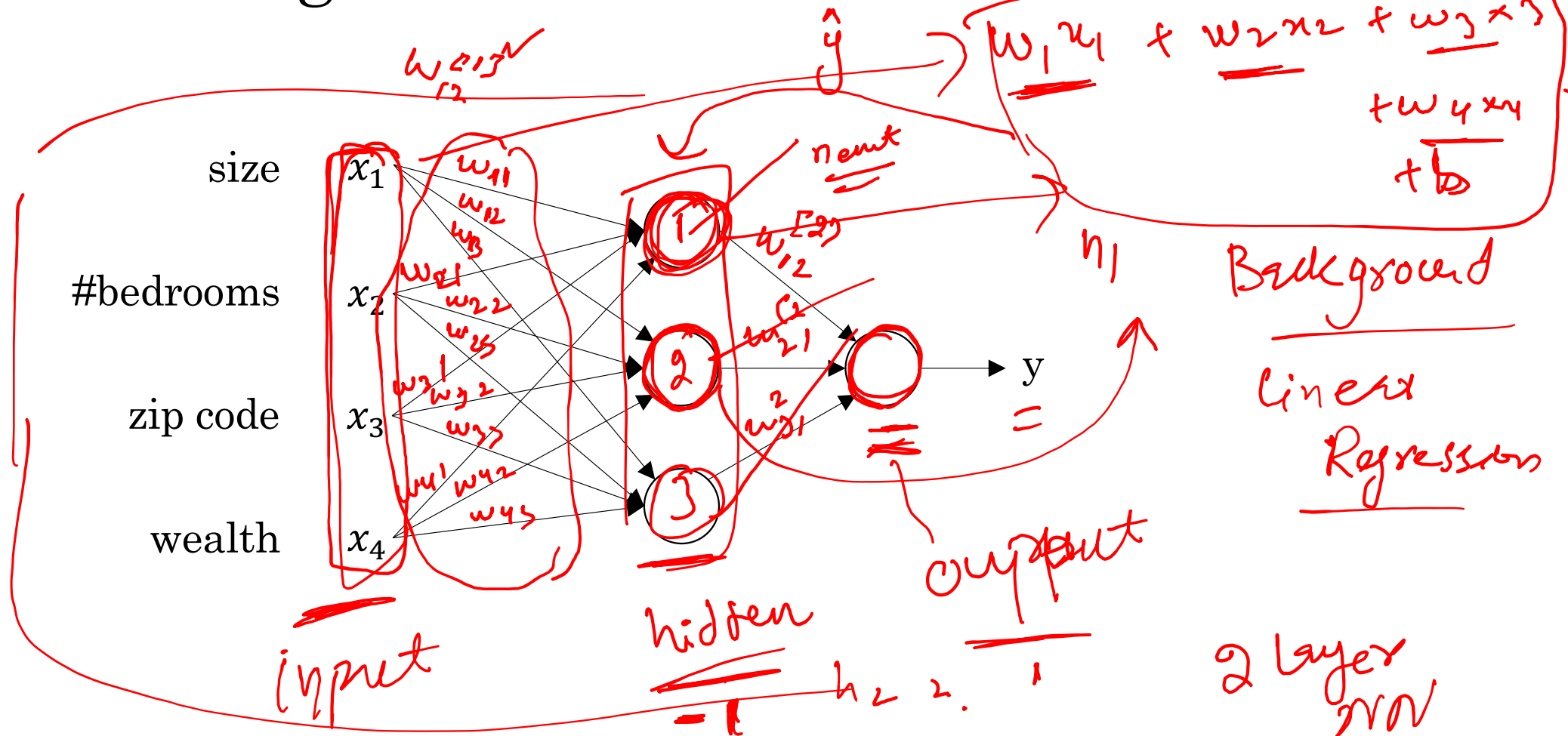
Introduction to Deep Learning

What is a Neural Network?

Housing Price Prediction



Housing Price Prediction





deeplearning.ai

Introduction to Deep Learning

Supervised Learning with Neural Networks

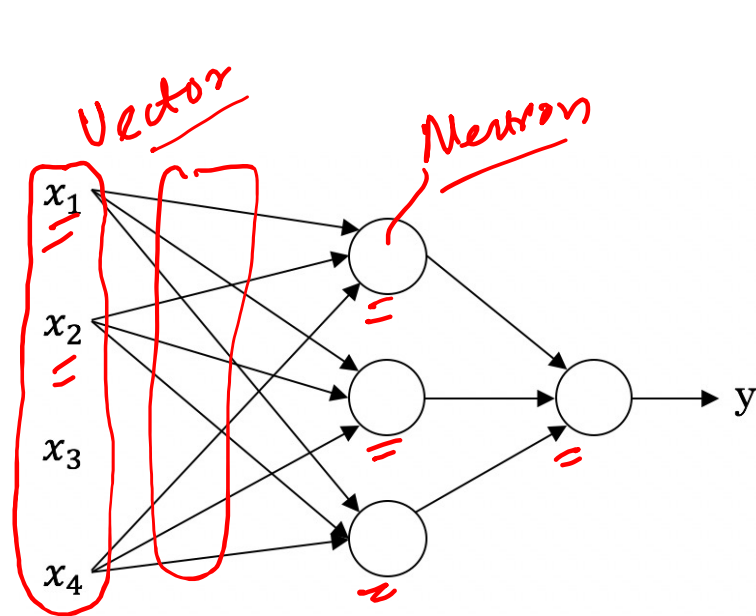
Supervised Learning

Input(x)	Output (y)	Application
Home features	Price	Real Estate
Ad, user info	Click on ad? (0/1)	Online Advertising
Image	Object (1,...,1000)	Photo tagging
Audio	Text transcript	Speech recognition
English	Chinese	Machine translation
Image, Radar info	Position of other cars	Autonomous driving

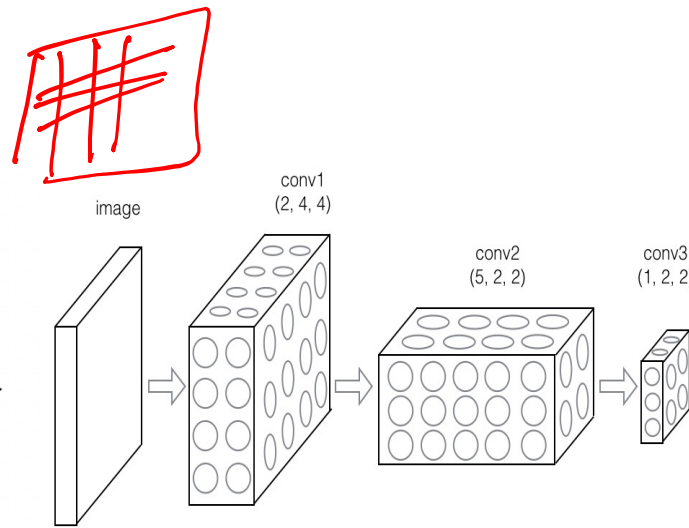
Continuous

Regression

Neural Network examples

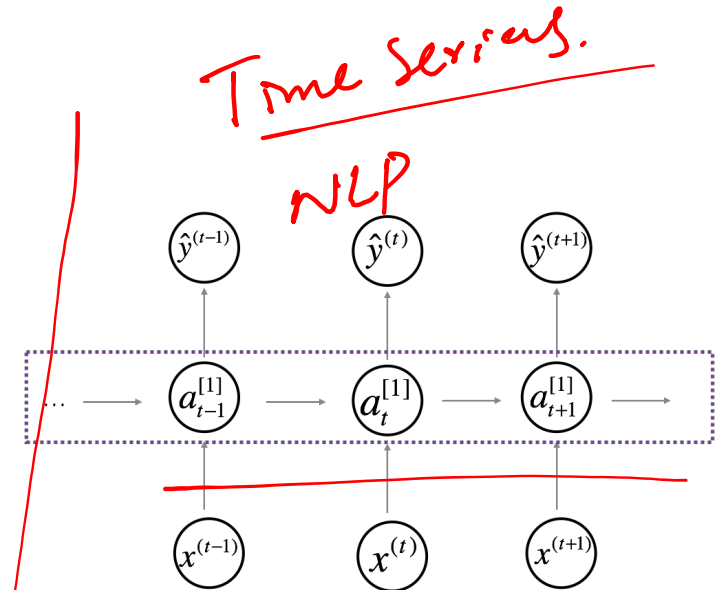


Standard NN



CNN

Convolutional NN



Transformers

Recurrent NN

LSTM RNN GRU

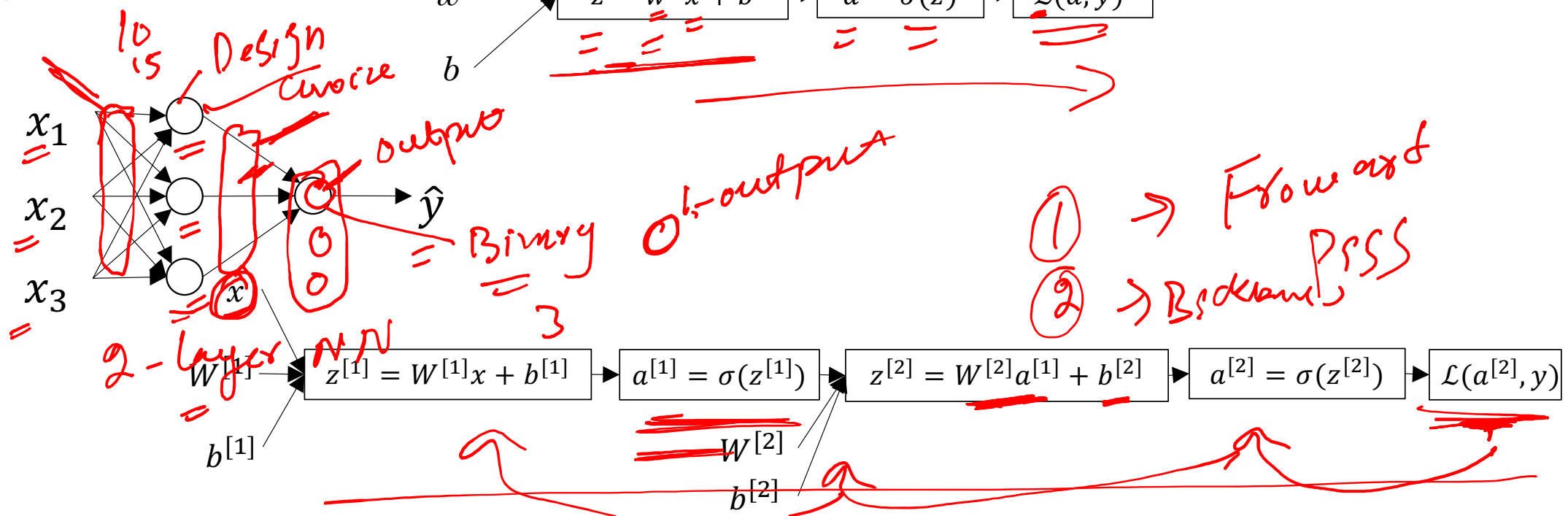
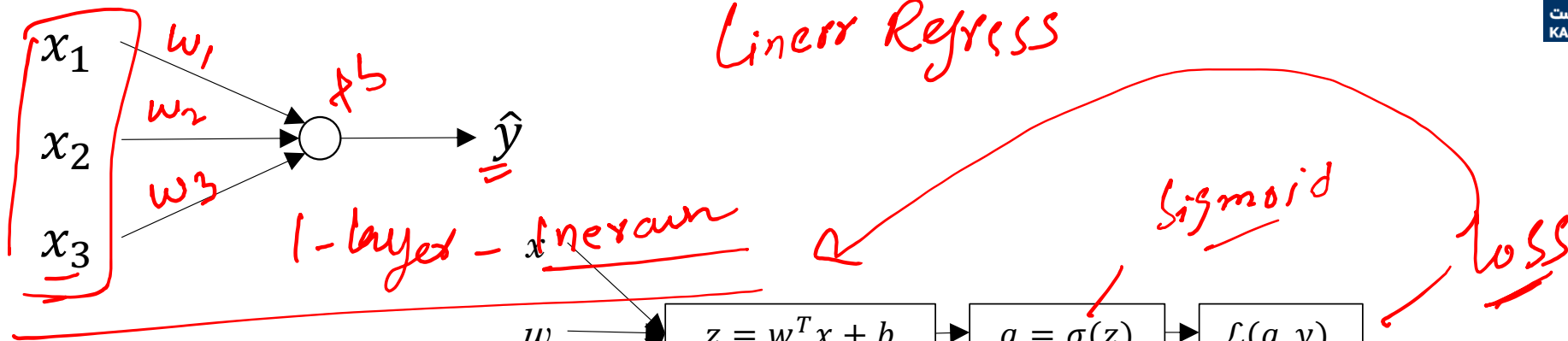


deeplearning.ai

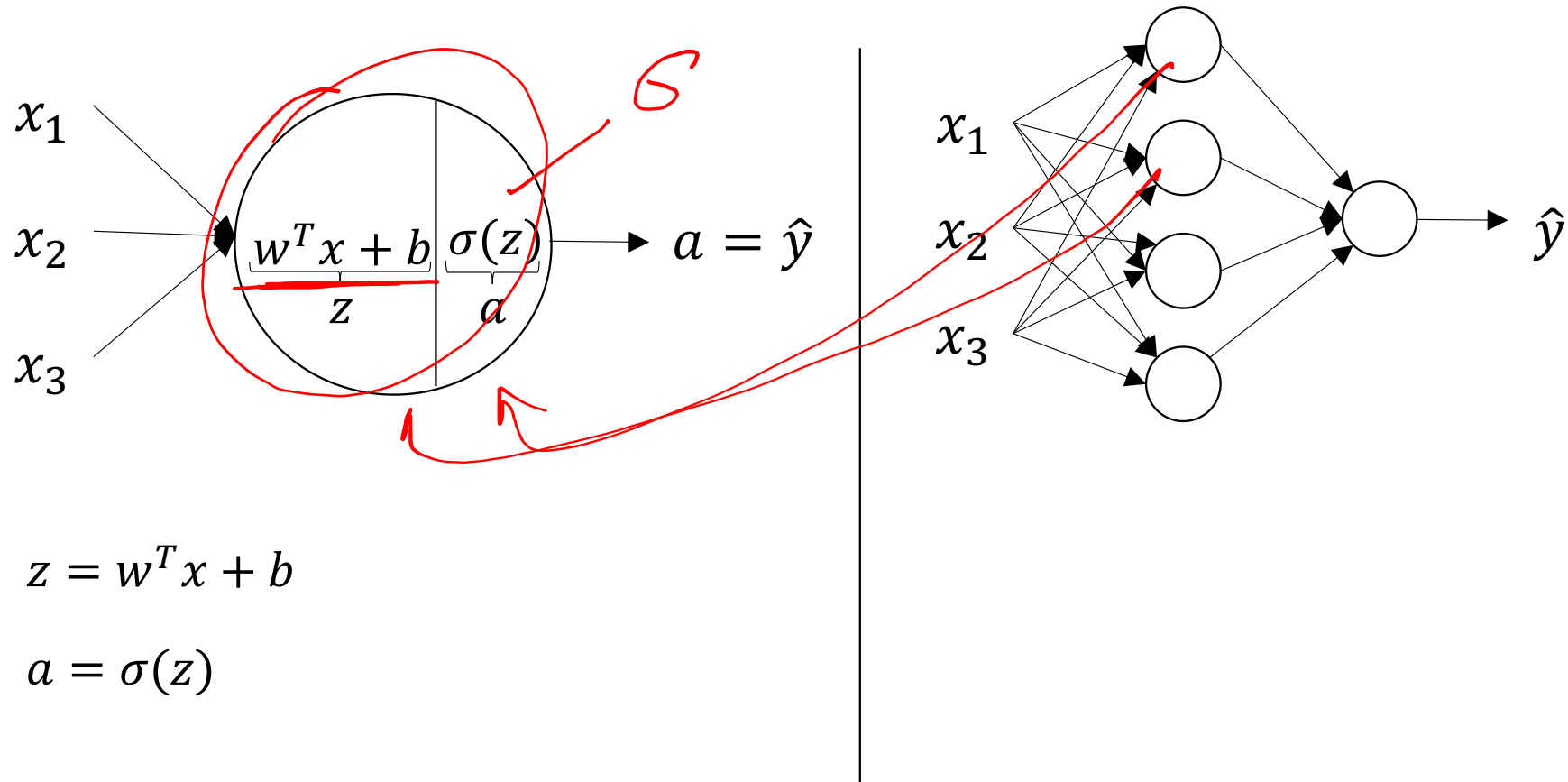
One hidden layer Neural Network

Neural Networks Overview

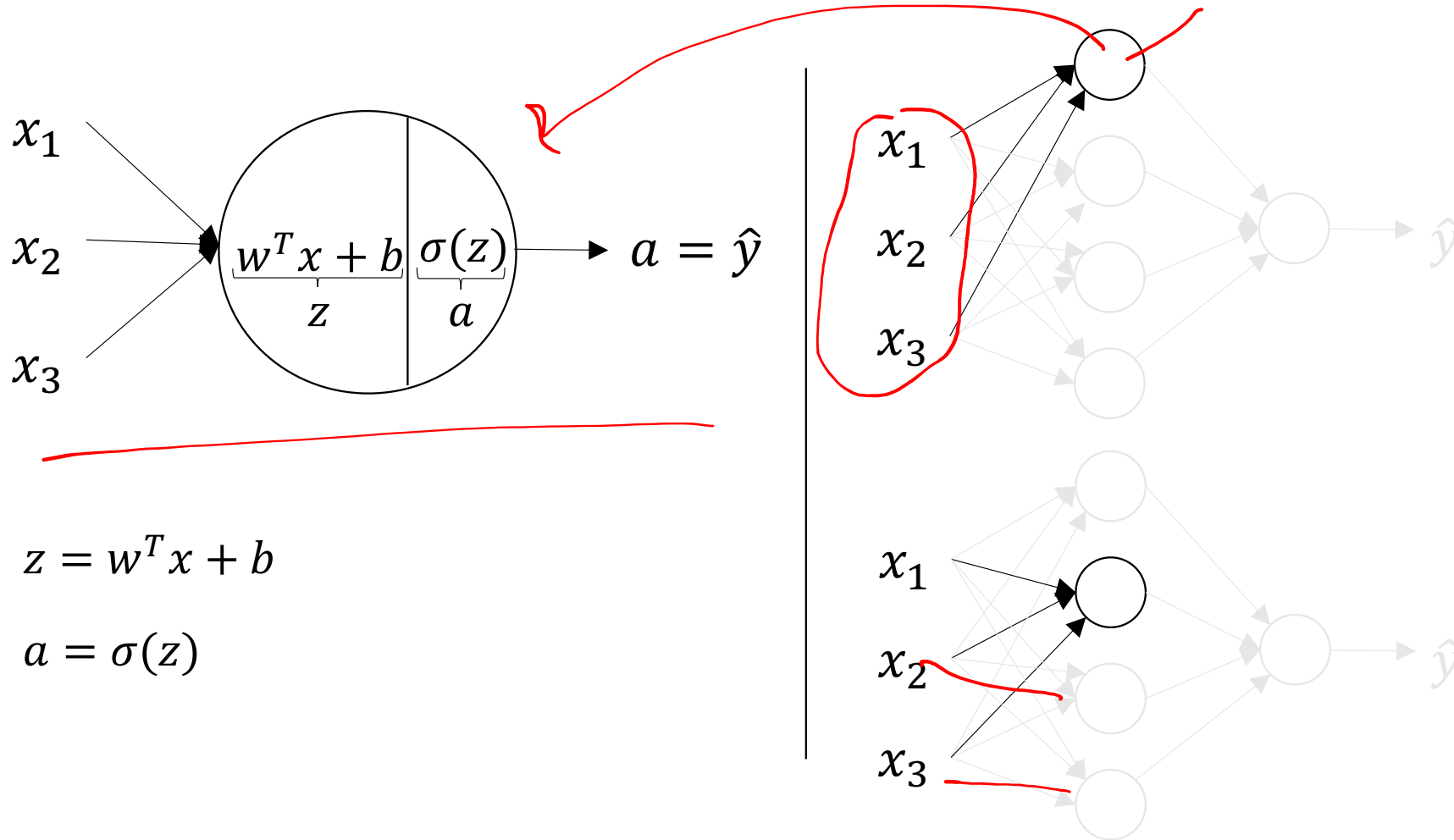
What is a Neural Network?



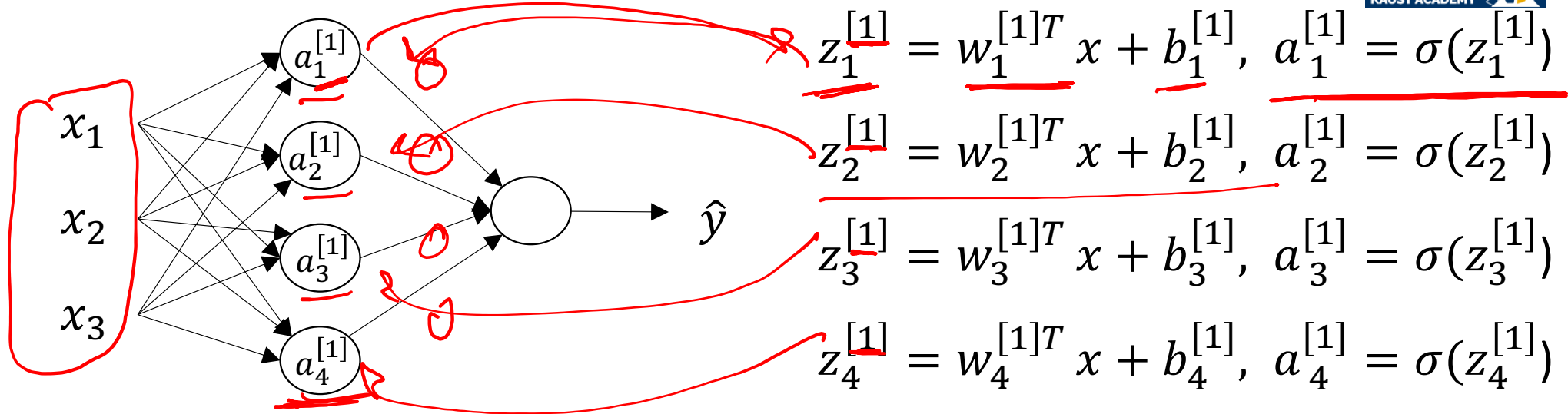
Neural Network Representation



Neural Network Representation



Neural Network Representation



2-layer
5-layer

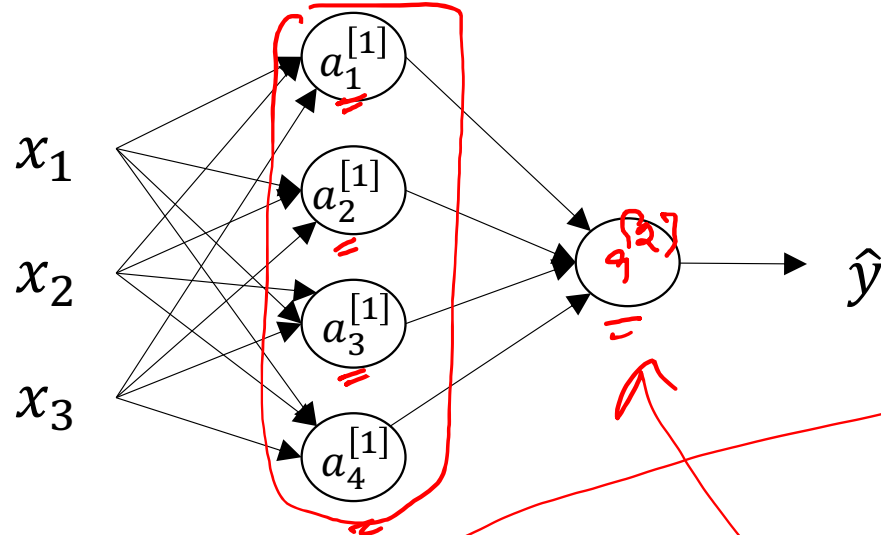
(n)

Design

$$z_1^{[2]} = c \quad \text{---} \quad \sigma(\quad)$$

$\hat{y}$

Neural Network Representation learning



Given input x :

$$z^{[1]} = \underline{W^{[1]}}x + b^{[1]}$$

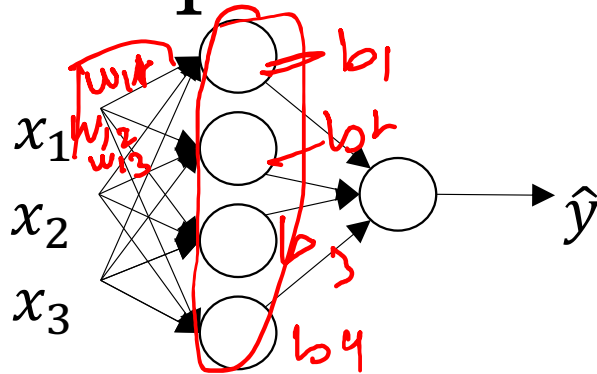
$$\underline{a^{[1]}} = \sigma(z^{[1]})$$

$$z^{[2]} = W^{[2]}a^{[1]} + b^{[2]}$$

$$a^{[2]} = \sigma(z^{[2]})$$

$$\begin{bmatrix} a_1 \\ a_2 \\ a_3 \\ a_4 \end{bmatrix} = \underline{a^{[1]}}$$

Recap of vectorizing across multiple examples



for $i = 1$ to m

$$z^{[1]}(i) = \underline{W^{[1]}} x^{(i)} + b^{[1]}$$

$$a^{[1]}(i) = \sigma(z^{[1]}(i))$$

$$z^{[2]}(i) = W^{[2]} a^{[1]}(i) + b^{[2]}$$

$$a^{[2]}(i) = \sigma(z^{[2]}(i))$$

$$\underline{X} = \begin{bmatrix} | & | & | & | \\ x^{(1)} & x^{(2)} & \dots & x^{(m)} \\ | & \dots & | & - \\ \hline \end{bmatrix}$$

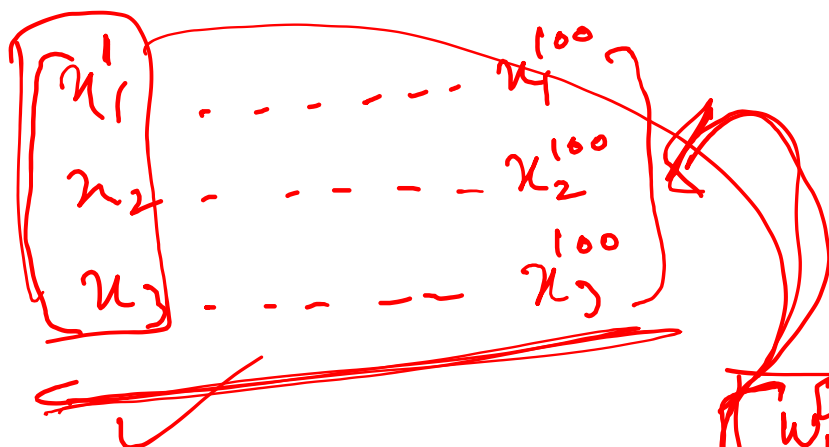
$$\underline{A^{[1]}} = \begin{bmatrix} | & | & | & | \\ a^{[1]}(1) & a^{[1]}(2) & \dots & a^{[1]}(m) \\ | & \dots & | & - \\ \hline \end{bmatrix}$$

$$\underline{Z^{[1]}} = \underline{W^{[1]}} \underline{X} + b^{[1]} \quad \text{--- } \underline{\text{Line}}$$

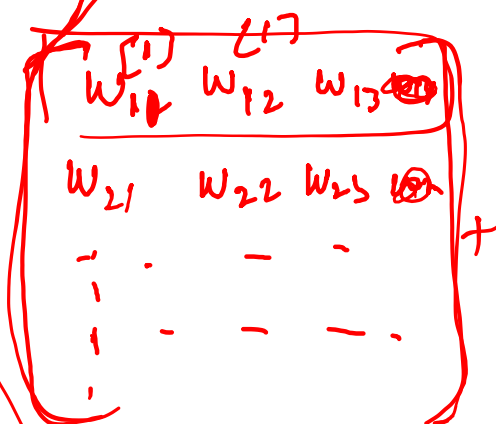
$$A^{[1]} = \sigma(Z^{[1]})$$

$$Z^{[2]} = W^{[2]} A^{[1]} + b^{[2]}$$

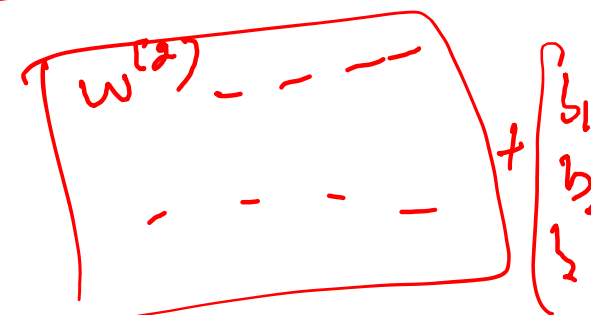
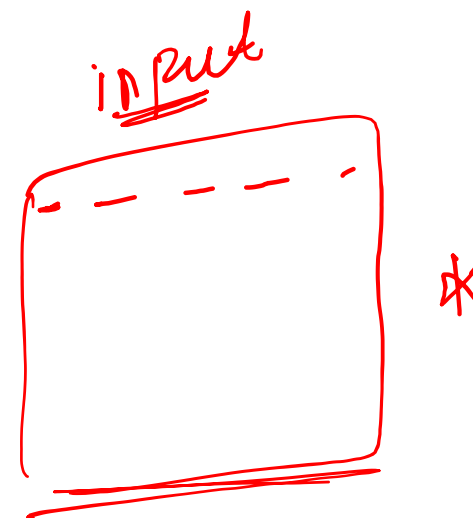
$$A^{[2]} = \sigma(Z^{[2]})$$



(a)



$$\begin{bmatrix} b_1 \\ b_2 \\ b_3 \\ b_4 \end{bmatrix} = z$$



Gradient descent for neural networks

Parameters $w^{[1]}, b^{[1]}, w^{[2]}, b^{[2]}$

$J(w^{[1]}, b^{[1]}, w^{[2]}, b^{[2]})$

$\frac{\partial J}{\partial w^{[1]}}$, $\frac{\partial J}{\partial b^{[1]}}$, $\frac{\partial J}{\partial w^{[2]}}$, $\frac{\partial J}{\partial b^{[2]}}$



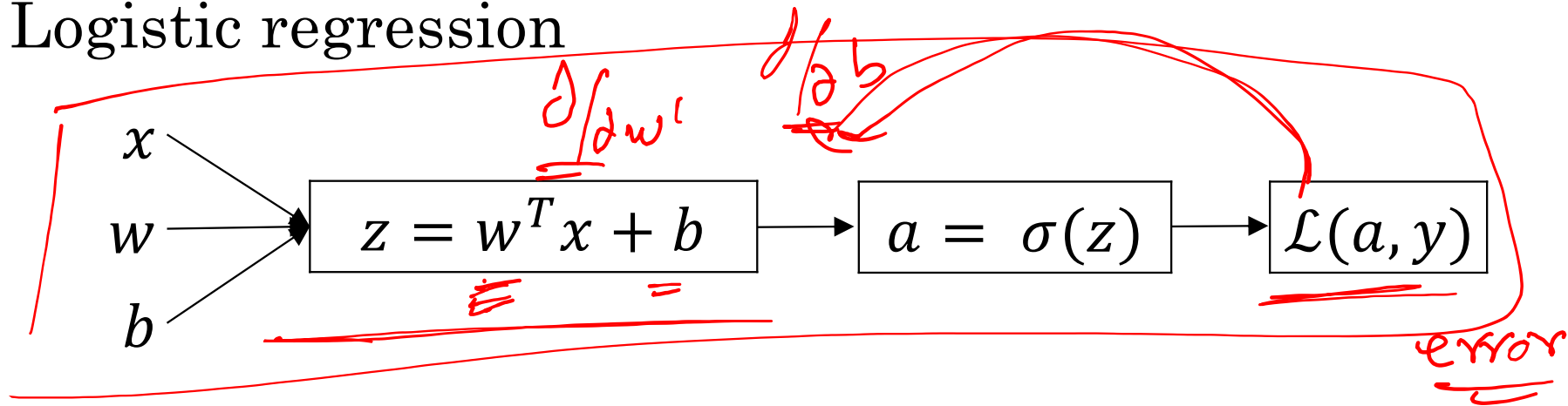
deeplearning.ai

One hidden layer Neural Network

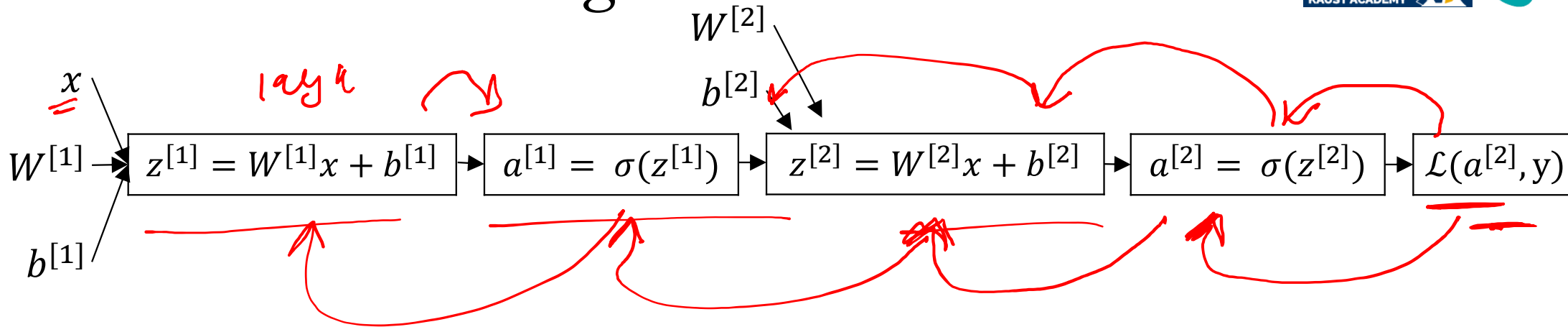
Backpropagation intuition

Computing gradients

Logistic regression



Neural network gradients



Summary of gradient descent



$$\underline{dz^{[2]}} = \underline{a^{[2]}} - \underline{y}$$

$$\underline{dW^{[2]}} = \underline{dz^{[2]}} \underline{a^{[1]}}^T$$

$$\underline{db^{[2]}} = \underline{dz^{[2]}}$$

$$\underline{dz^{[1]}} = \underline{W^{[2]}}^T \underline{dz^{[2]}} * \underline{g^{[1]}}'(z^{[1]})$$

$$\underline{dW^{[1]}} = \underline{dz^{[1]}} x^T$$

$$\underline{db^{[1]}} = \underline{dz^{[1]}}$$

Summary of gradient descent



$$dz^{[2]} = \underline{a}^{[2]} - y$$

$$dW^{[2]} = dz^{[2]} a^{[1]T}$$

$$db^{[2]} = dz^{[2]}$$

$$dz^{[1]} = W^{[2]T} dz^{[2]} * g^{[1]'}(z^{[1]})$$

$$dW^{[1]} = dz^{[1]} x^T$$

$$db^{[1]} = dz^{[1]}$$

$$dZ^{[2]} = \underline{\underline{A}}^{[2]} - Y$$

$$dW^{[2]} = \frac{1}{m} dZ^{[2]} A^{[1]T}$$

$$db^{[2]} = \frac{1}{m} \text{np.sum}(dZ^{[2]}, \text{axis} = 1, \text{keepdims} = \text{True})$$

$$dZ^{[1]} = W^{[2]T} dZ^{[2]} * g^{[1]'}(Z^{[1]})$$

$$dW^{[1]} = \frac{1}{m} dZ^{[1]} X^T$$

$$db^{[1]} = \frac{1}{m} \text{np.sum}(dZ^{[1]}, \text{axis} = 1, \text{keepdims} = \text{True})$$

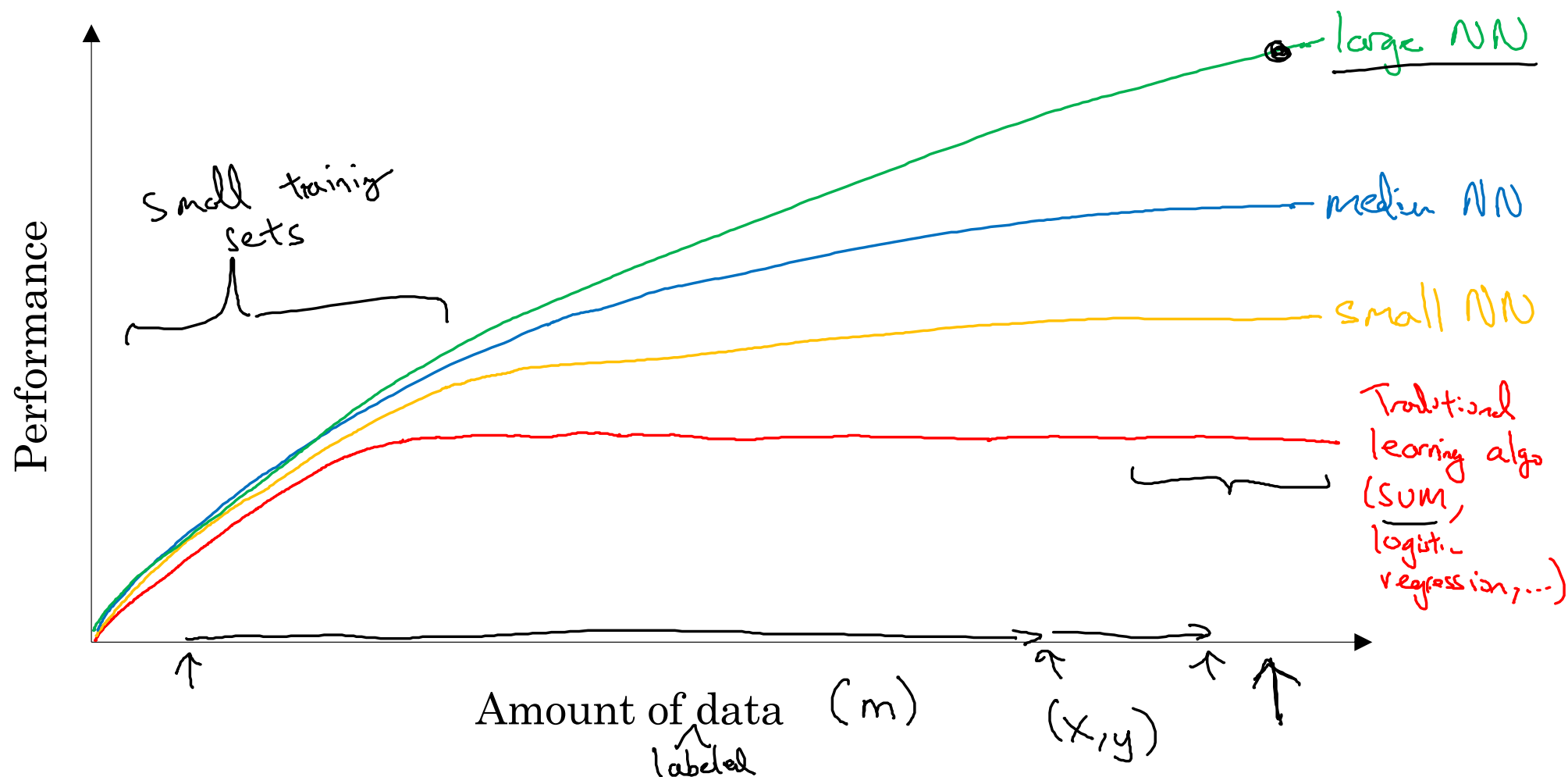


deeplearning.ai

Introduction to Neural Networks

Why is Deep Learning taking off?

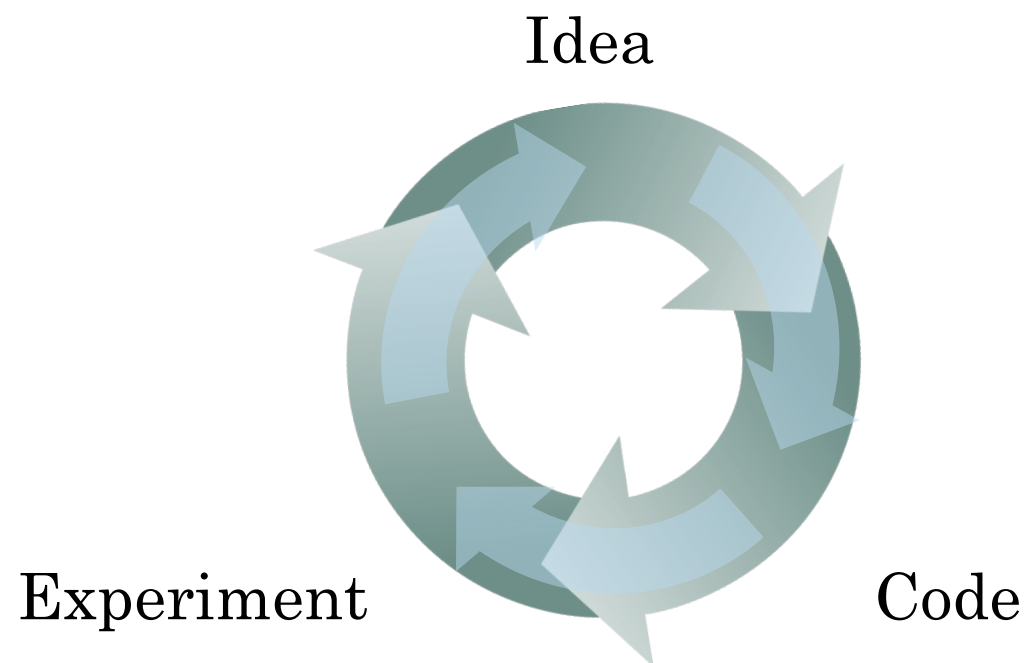
Scale drives deep learning progress



Scale drives deep learning progress



- Data
- Computation
- Algorithms



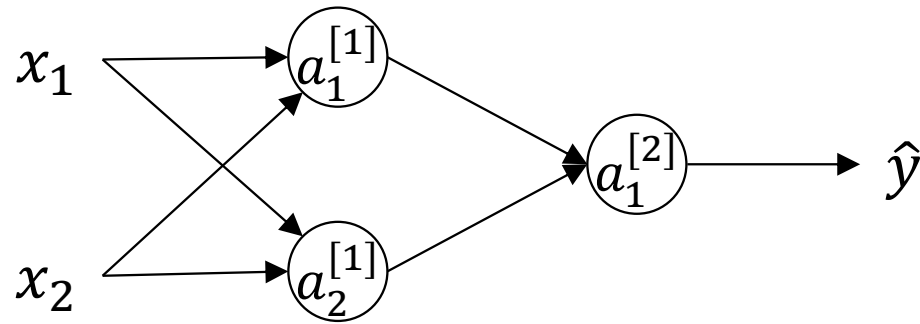


deeplearning.ai

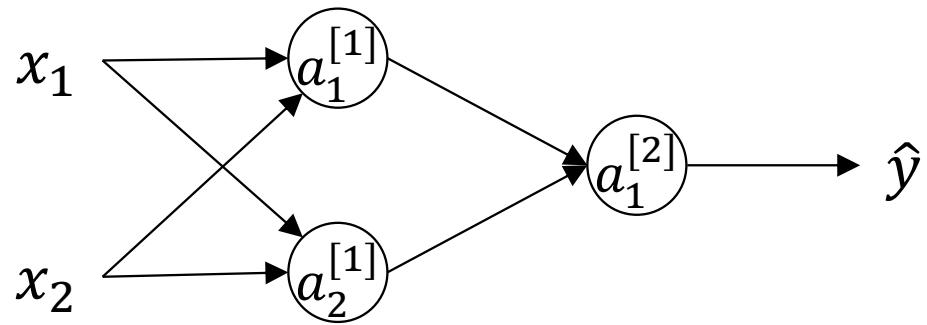
One hidden layer Neural Network

Random Initialization

What happens if you initialize weights to zero?



Random initialization



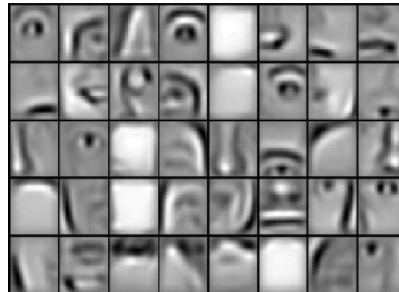
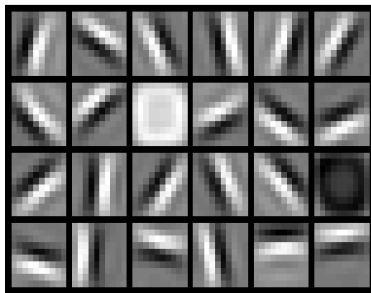
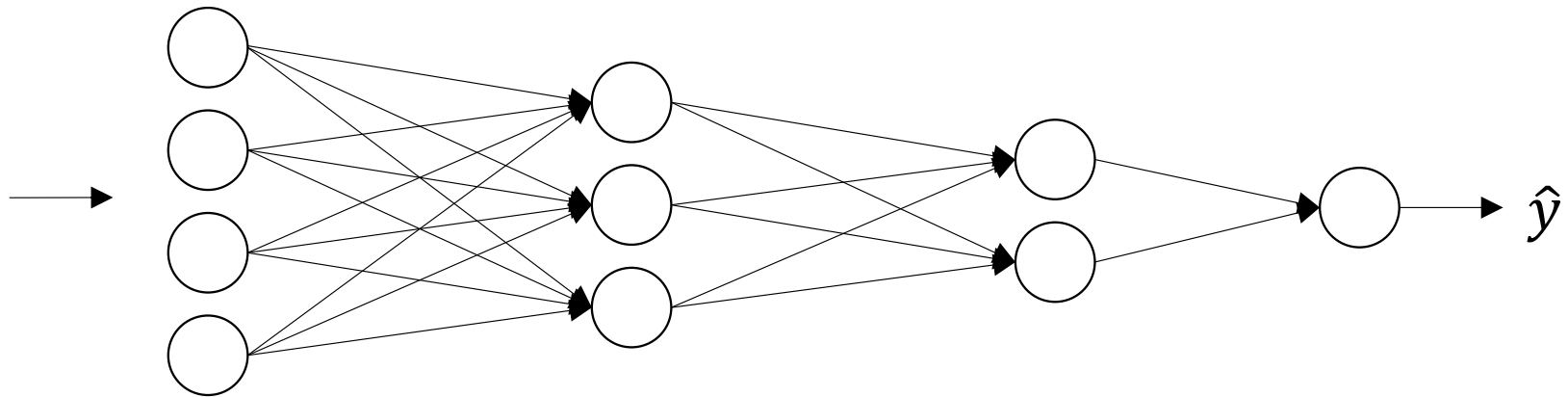


deeplearning.ai

Deep Neural Networks

Why deep representations?

Intuition about deep representation



Circuit theory and deep learning

Informally: There are functions you can compute with a “small” L -layer deep neural network that shallower networks require exponentially more hidden units to compute.

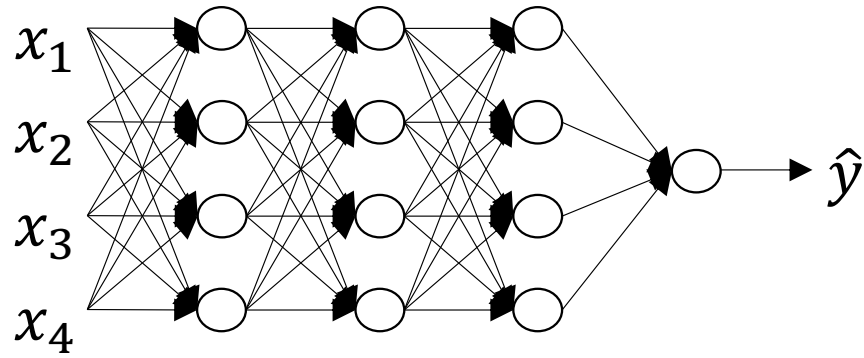


deeplearning.ai

Deep Neural Networks

Building blocks of
deep neural networks

Forward and backward functions



Forward and backward functions





deeplearning.ai

Deep Neural Networks

Forward and backward propagation

Forward propagation for layer l

Input $a^{[l-1]}$

Output $a^{[l]}$, cache $(z^{[l]})$



Backward propagation for layer l

Input $da^{[l]}$

Output $da^{[l-1]}, dW^{[l]}, db^{[l]}$



Summary

